Design & Analysis of Algorithms

Lab 1: Algorithm Performance Analysis Workbench

Name: Kaushal
Roll No: 2501730085
Email: 2501730085@krmu.edu.in

Q1. 1.2 Linear Search and Binary Search Comparison (5 marks)

```
def compare_search_algorithms(arr, target):
  linear_index = -1
  linear_comparisons = 0

  for i in range(len(arr)):
    linear_comparisons += 1
    if arr[i] == target:
      linear_index = i
      break
  low = 0
  high = len(arr) -1
  binary_index = -1
  binary_comparisons = 0

  while low <= high:
    mid = (low + high)//2
    binary_comparisons += 1

    if arr[mid] == target:
      binary_index = mid
      high = mid -1
    elif arr[mid] < target:
      low = mid + 1
    else:
      high = mid - 1

  if linear_comparisons < binary_comparisons:
    Better = "Linear Search"
  elif binary_comparisons < linear_comparisons:
    Better = "Binary Search"
  else:
    Better = "Both Equal"

  return[
    "Search Comparison Report",
    "Linear Search",
    f"Index: {linear_index}",
    f"Comparisons: {linear_comparisons}",
    "Binary Search",
    f"Index: {binary_index}",
    f"Comparisons: {binary_comparisons}",
    f"Better Algorithm: {Better}"

  ]
  return
```

Q2. 1.3 Bubble Sort and Insertion Sort Performance Comparison (10 marks)

```
def insertion_sort(arr):
  comparisons = 0
  shifts = 0

  for i in range(1,len(arr)):
    key = arr[i]
    j = i - 1

    while j >= 0:
      comparisons += 1

      if arr[j] > key:
        arr[j + 1] = arr[j]
        shifts += 1
        j -= 1
      else:
        break
    arr[j + 1] = key
  return arr, comparisons, shifts
```

```
def bubble_sort(arr):
  comparisons = 0
  swaps = 0

  for i in range(len(arr) - 1):
    swapped = False

    for j in range(len(arr) -i -1):
      comparisons += 1

      if arr[j] > arr[j+1]:
        swaps += 1
        swapped = True
        arr[j], arr[j + 1] = arr[j + 1],arr[j]

    if swapped==False:
      break
  return arr, comparisons, swaps

def add_dataset_report(result, dataset_name, arr):
  bubble_arr = arr.copy()
  insertion_arr = arr.copy()

  bubble_sorted, bubble_comparisons, bubble_swaps = bubble_sort(bubble_arr)

  insertion_sorted, insertion_comparisons, insertion_shifts = insertion_sort(insertion_arr)

  if bubble_comparisons < insertion_comparisons:
    better_algorithm = "Bubble Sort"
  elif insertion_comparisons < bubble_comparisons:
    better_algorithm = "Insertion Sort"
  else:
    better_algorithm = "Both Equal"
  result.append(dataset_name)
  result.append("Bubble Sorted: " + " ".join(map(str, bubble_sorted)))
  result.append("Bubble Comparisons: " + str(bubble_comparisons))
  result.append("Bubble Swaps: " + str(bubble_swaps))
  result.append("Insertion Sorted: " + " ".join(map(str, insertion_sorted)))
  result.append("Insertion Comparisons: " + str(insertion_comparisons))
  result.append("Insertion Shifts: " + str(insertion_shifts))
  result.append("Better Algorithm: " + better_algorithm)



def compare_bubble_insertion(random_data, sorted_data, reverse_data):
  result = []
  result.append("Sorting Performance Report")

  add_dataset_report(result, "Random Dataset", random_data)
  add_dataset_report(result, "Sorted Dataset", sorted_data)
  add_dataset_report(result, "Reverse Dataset", reverse_data)
  return result
```